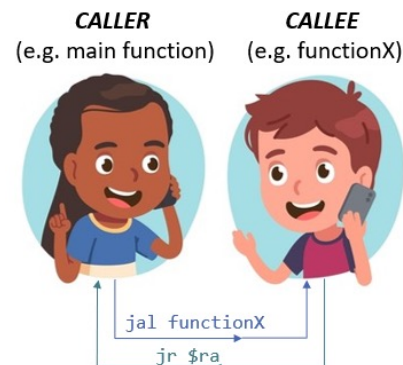
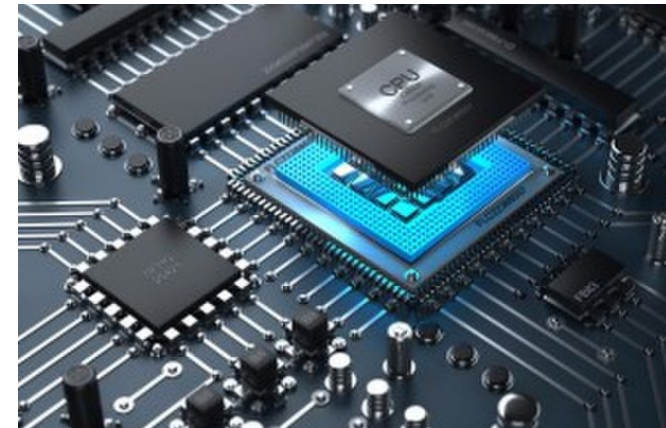


```
loop: lw    $t3, 0($t0)
      lw    $t4, 4($t0)
      add   $t2, $t3, $t4
      sw    $t2, 8($t0)
      addi  $t0, $t0, 4
      addi  $t1, $t1, -1
      bgtz  $t1, loop
```

Assembler

```
0x8d0b0000
0x8d0c0004
0x016c5020
0xad0a0008
0x21080004
0x2129ffff
0x1d20fff9
```



Lecture 11:

The Stack & the MIPS Calling Convention

CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA



Administrative

Current homework due TODAY!

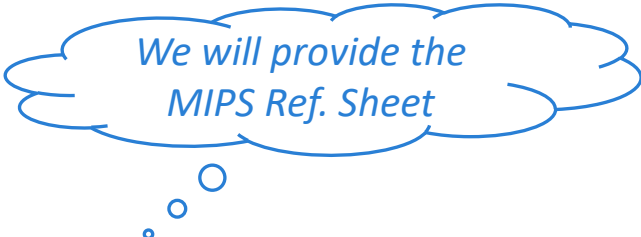
New homework will be out on Canvas soon!

New readings for this week!

- **Handout 5** on Functions and the MIPS Calling Convention
-

Worksheet for this lecture available on Canvas

Reminder: Office Hours R 4 U!!!!



*We will provide the
MIPS Ref. Sheet*

Quiz #3 is on Wednesday!

We will start it at 11 AM SHARP!!!

- Material is from Lectures **8, 9, 10**, and **11**, and the **Reading: CS64_Handouts 3, 4**

Function Calls Within Functions...

```
void foo() {  
    bar();  
}  
void bar() {  
    baz();  
}  
void baz() {}
```

Given what we've said so far... consider the code shown here.....>>

What about this code makes our previously discussed setup *not work*?

ANS: You would need **multiple copies of \$ra**

You'd have to copy the value of **\$ra** somewhere before calling another function

- Where, though? Other registers?? That's likely NOT ok... (*why?*)

So obviously, you have to use something else... in memory... iiiinnnn...

The STACK!



Solution??!!

Store certain data in this “**call stack**” in memory only when needed

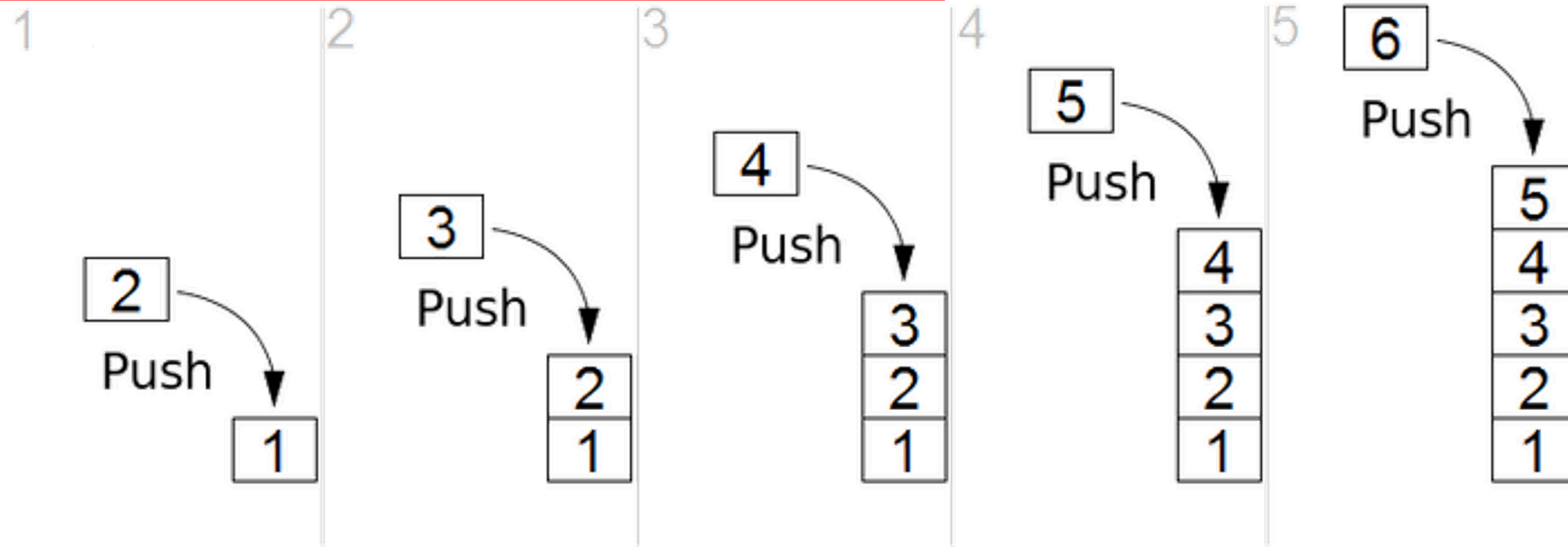
Sounds like a lot for a programmer to keep track of!

- Yes, that’s why the compiler usually does this for you in HLL like C++!

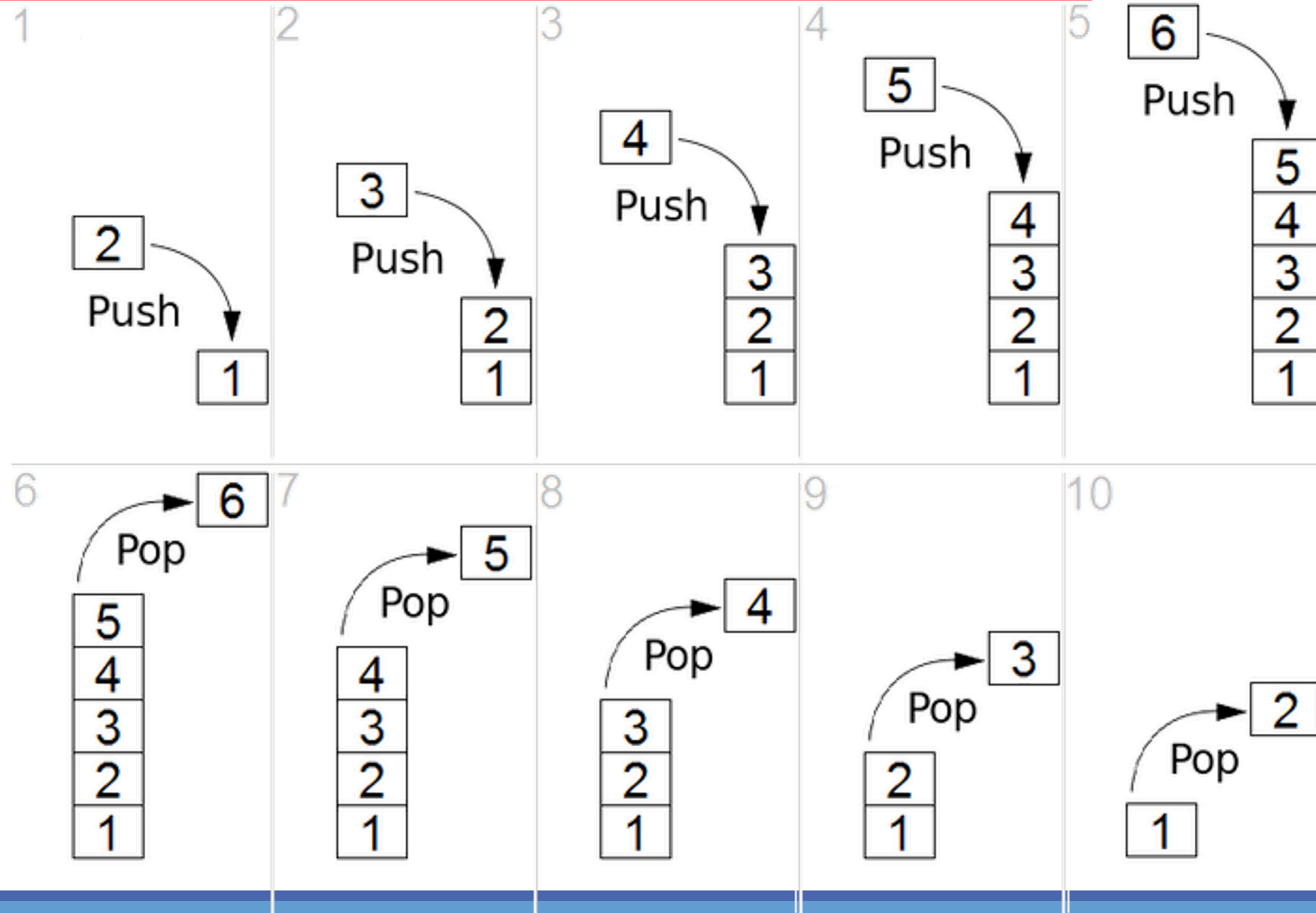
We have to learn the *rules* for how to do this at the CPU level

- But let’s get the basics down first...

STACK (LIFO) OPERATIONS: **PUSH**



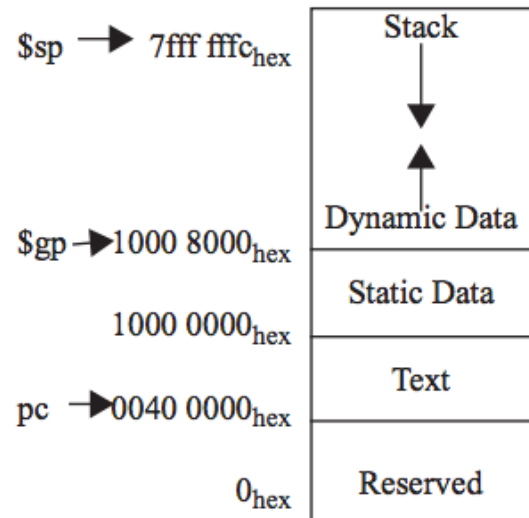
STACK (LIFO) OPERATIONS: **PUSH** AND **POP**



The Stack

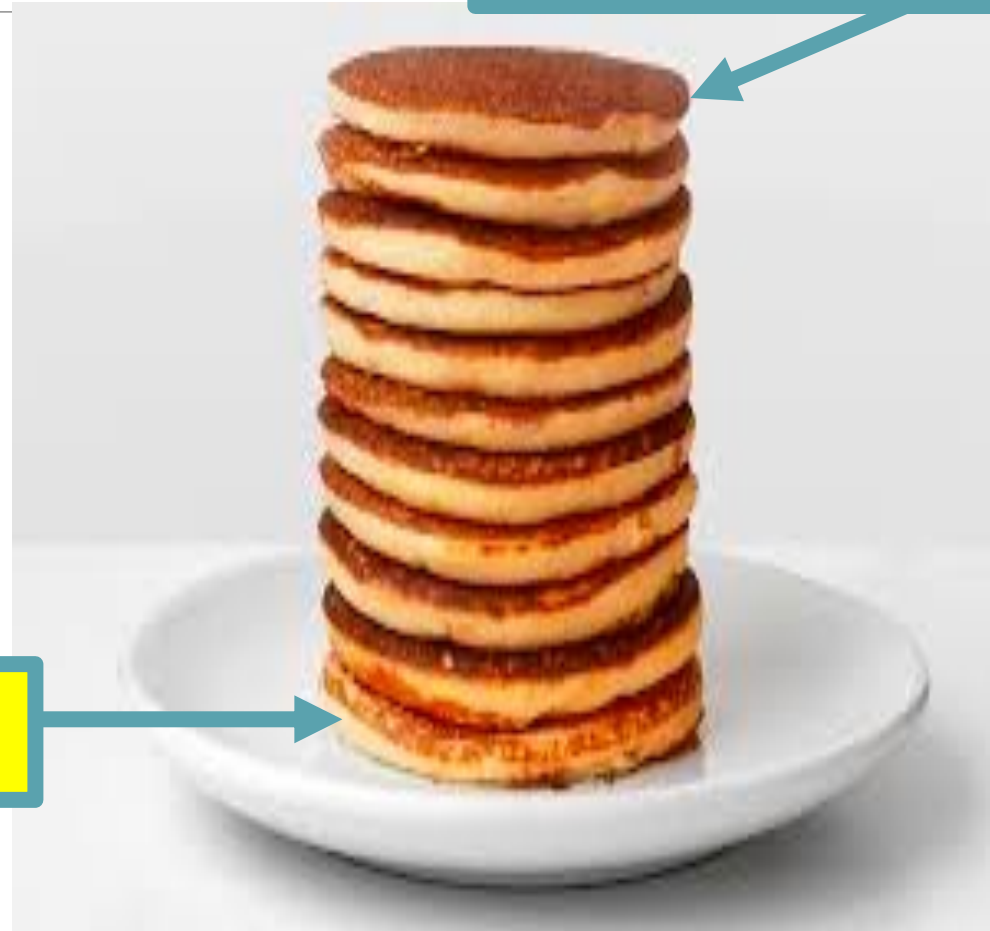
When a program starts executing instructions, a certain *contiguous* section of memory is set aside for the program called the stack.

MEMORY ALLOCATION



Bottom of the stack

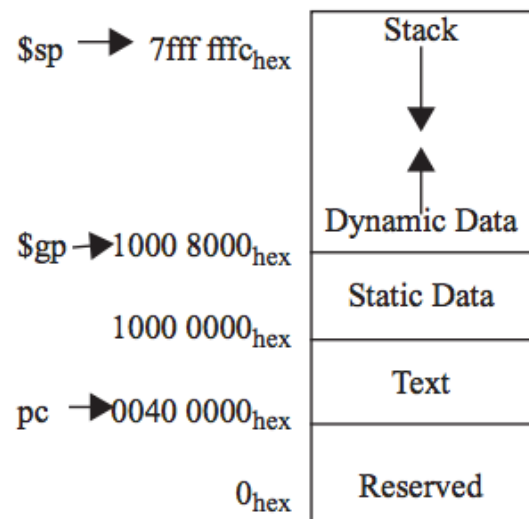
Top of the stack



The Stack

When a program starts executing instructions, a certain *contiguous* section of memory is set aside for the program called the stack.

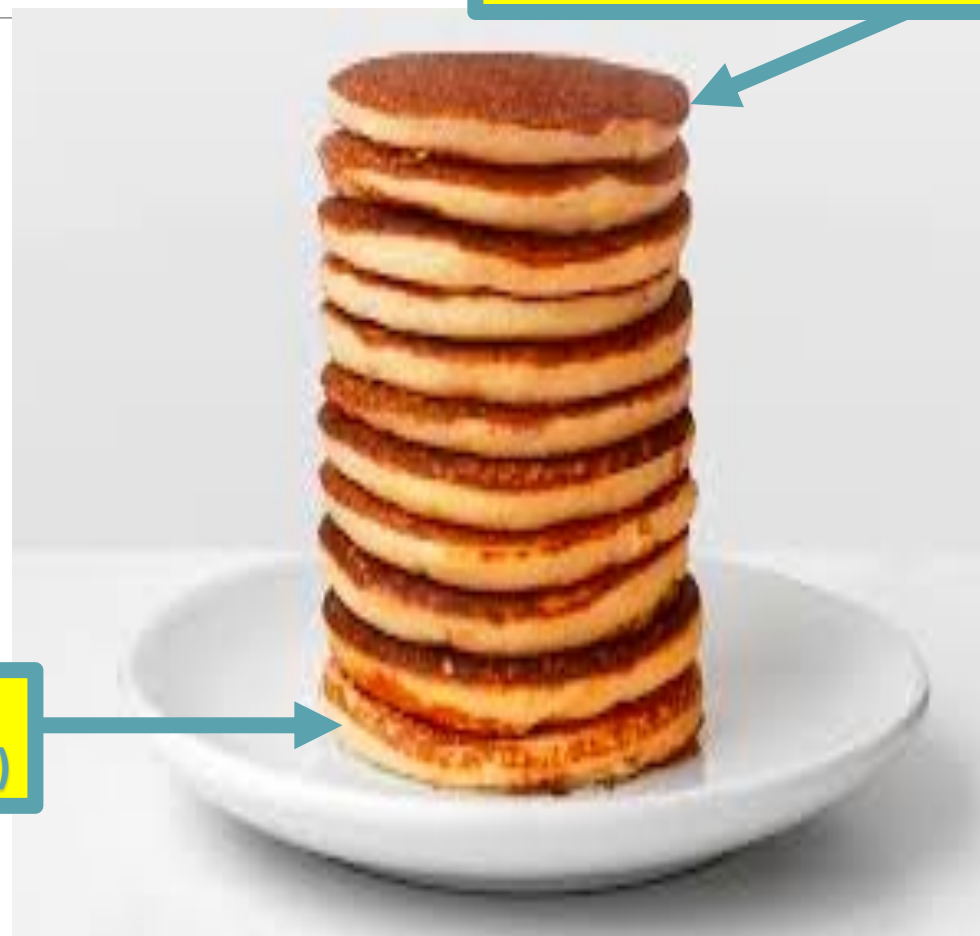
MEMORY ALLOCATION



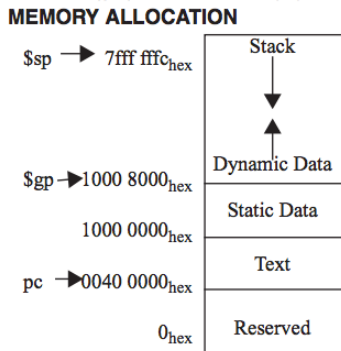
Bottom of the stack
(Higher address in MIPS)

\$7FFF FFFC

Top of the stack
(Lower address in MIPS)



The Stack



The **stack pointer** is a register (**\$sp**) that contains the address of the **top of the stack**.

The *bottom* of the stack is where the stack starts grow *initially*

- In MIPS, that address is *fixed*: **0x7FFF FFFC** (see MIPS Ref. Card)
- It's the address value that is *initialized* in **\$sp** at the start of a program

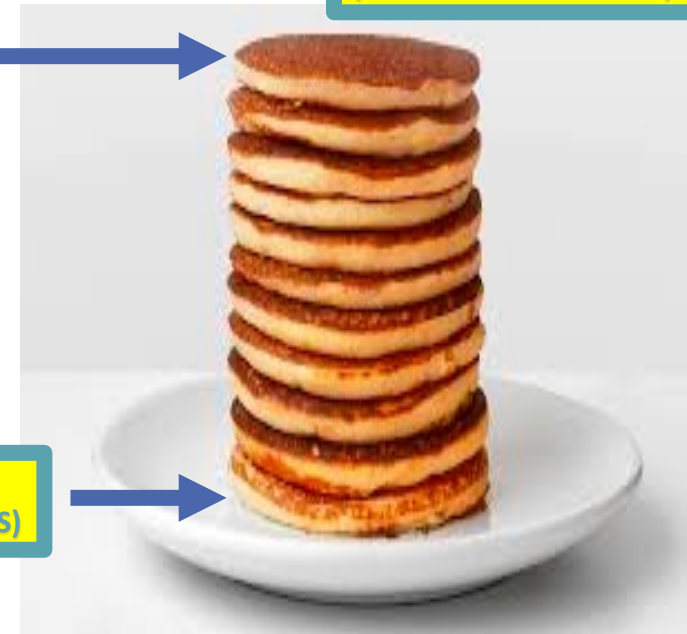
In MIPS, we **don't** grow the stack from smaller address to larger address numbers:

***we grow the stack from larger addresses to smaller address
(in other words, the pancake stack visual should be upside down!!)***

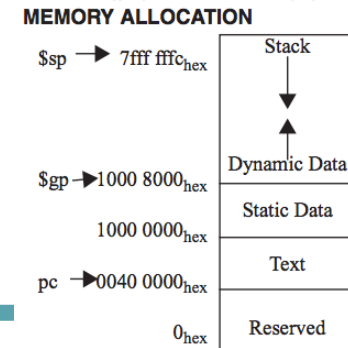
\$sp

Top of the stack
(Lower address in MIPS)

Bottom of the stack
(Higher address in MIPS)



The Stack



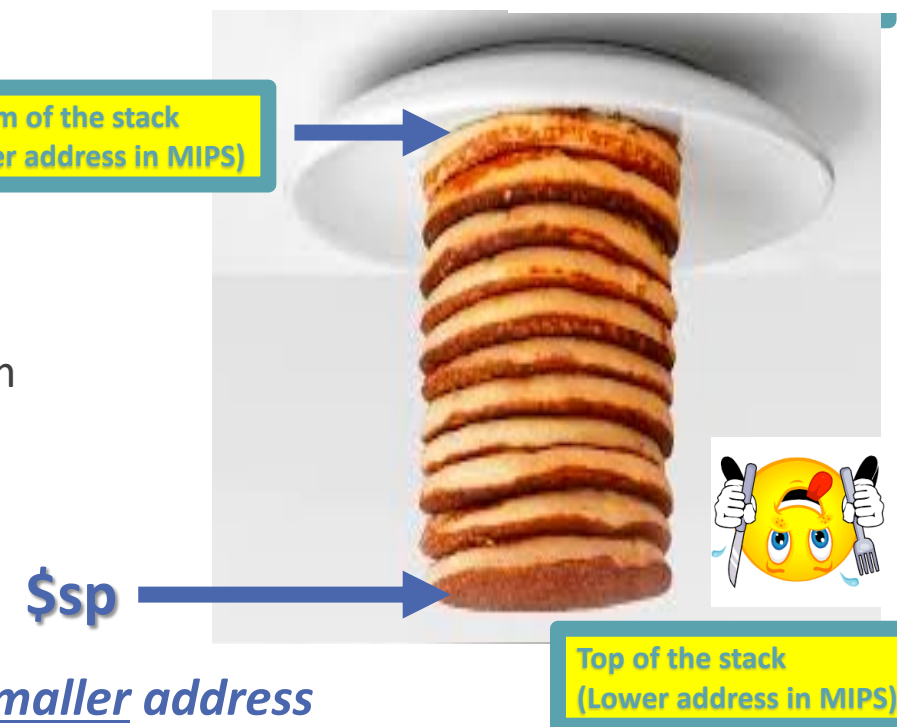
The **stack pointer** is a register (**\$sp**) that contains the address of the **top of the stack**.

The *bottom* of the stack is where the stack starts grow *initially*

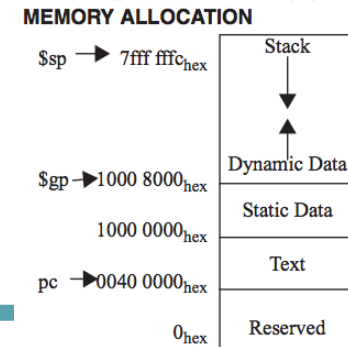
- In MIPS, that address is *fixed*: **0x7FFF FFFC** (see MIPS Ref. Card)
- It's the address value that is *initialized* in **\$sp** at the start of a program

In MIPS, we **don't** grow the stack from smaller address to larger address numbers:

***we grow the stack from larger addresses to smaller address
(in other words, the pancake stack visual should be upside down!!)***



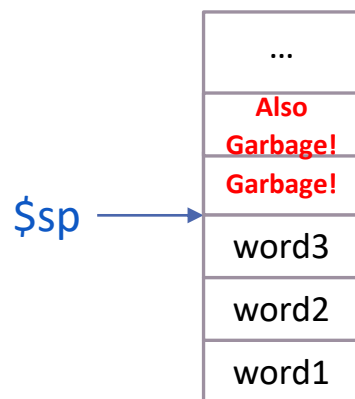
The Stack



EXAMPLE: let's say the **\$sp** is **0xFFC** bytes away from the "bottom" *(first of all, let's ask: is this a "valid" amount?)*

This means that $\$sp = 0x7FFF\ FFFC - 0xFFC = 0x7FFF\ F000$

- Did you notice that we **SUBTRACTED** that amount from the "bottom"??!?
- Note that any mem data "above" the stack pointer is **not in the stack, by definition**

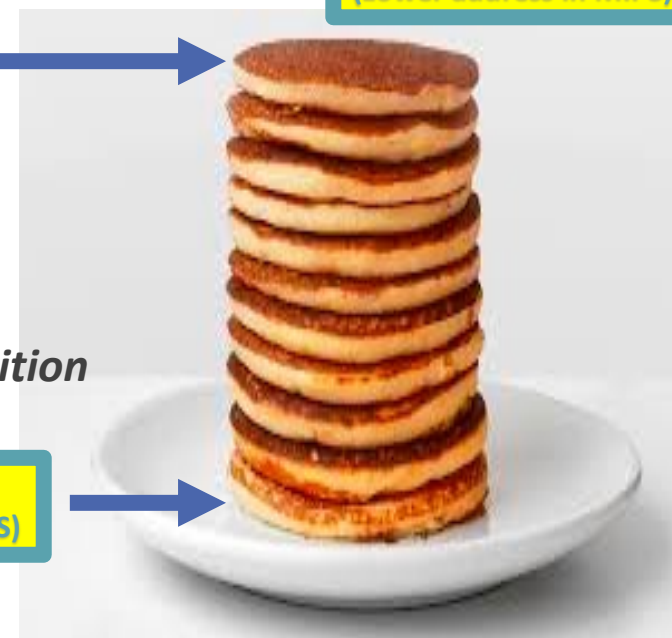


Remember, in MIPS, "above the \$sp", weirdly means it has an address *smaller* than \$sp

\$sp

Top of the stack
(Lower address in MIPS)

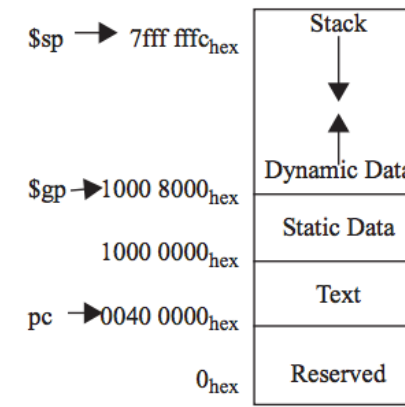
Bottom of the stack
(Higher address in MIPS)



The Stack

The Stack and the Heap share a common area in MIPS memory and they grow “towards” each other

MEMORY ALLOCATION



Of course, the stack can only keep growing up to a point...!

... because: we *can* run out of stack memory

- If that happens, we hit the “**stack limit**” and we can then get a **stack overflow error**

Stack Limit (Definition):

- Smallest *valid* address of a stack, where if **\$sp** gets any smaller, we get a stack overflow error

PREVIOUSLY ON CS 64!!! ...

SKIP INTRO

When we call a function in C++, we have *expectations* of what happens to variable values

```
int MahFunkshun(int m, int &f) {  
    f = 2*f;  
    int x = m - f;  
    return x;  
}  
  
int main() {  
    int a = 5, b = 2, c;  
    int x = 99;  
    c = MahFunkshun(a, b);  
    cout << a << b << x << c;  
}
```

Variable **m** is *passed by value*, therefore any changes made to it in **MahFunkshun**, WILL NOT be observed in **main()**

– in other words, we say “variable **a** in **main()** is **preserved**.”

Variable **f** is *passed by reference*, therefore any changes made to it in **MahFunkshun**, WILL be observed in **main()**

– in other words, we say “variable **b** in **main()** is **not preserved**.”

Fitting in the MIPS Calling Convention

We answer the following question:

When we *call* a function in MIPS assembly, should we make *assumptions* of what *registers* should NOT change values in the program?

- Yes: those would be the **ra-register** and also all the **s-registers**

And when we call a function, can we make *assumptions* of what *registers* WILL/MIGHT change values when a function is run?

- Yes: those would be *all the other* registers – **t-registers**, **a-registers**, and **v-registers**

The New Rules for MIPS Calling Convention

1

Preserved Registers:

\$s0 - \$s7, and \$ra

Unpreserved Registers: **\$t0 - \$t9, \$a0 - \$a3, and \$v0 - \$v1**

1. Values held in **Preserved Regs** immediately before a function call **MUST be the same** immediately after the function returns

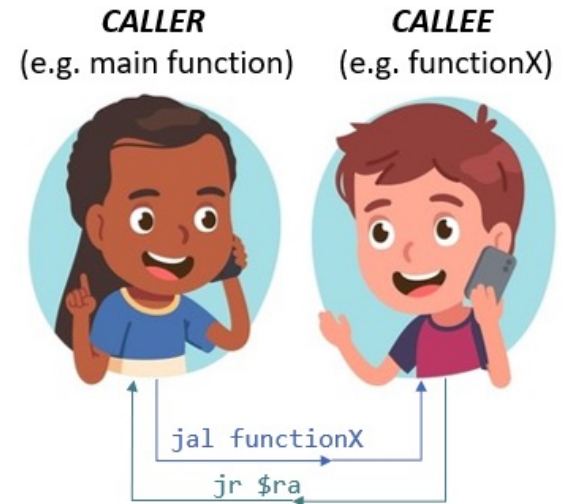
- What will “guarantee” this?
- How?
- All of them (all 9 of them?)
- And then?

*The function itself (the “**callee**”) will guarantee this*

*By **pushing** these registers into the stack right away*

*No! **Only the ones** the function uses (e.g., just \$s0 and \$ra)*

***Just before** it returns, the function **pops** these values back from the stack to the original registers*



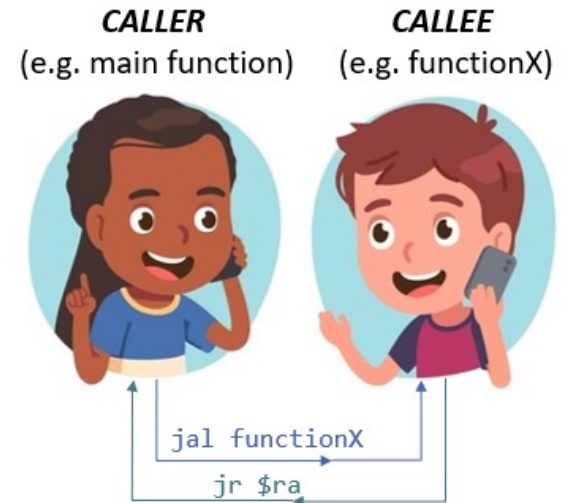
The New Rules for MIPS Calling Convention

2

Preserved Registers: `$s0 - $s7`, and `$ra`

Unpreserved Registers: `$t0 - $t9`, `$a0 - $a3`, and `$v0 - $v1`

2. Values held in **Unpreserved Regs** are NOT guaranteed to remain the same after the function call is performed.



- Who might “care” about this?
- And the function – does it care??
- What if the “**caller**” code NEEDS those register values for later?

*The part of the program that **calls** the function (the “**caller**”)*

No, the function “doesn’t care” about unpreserved regs!

*It’s the **caller’s** responsibility to “save” them somehow*

if it really needs to (and yes, they can save them onto the stack!)

BUT THE BEST STRATEGY IS: Copy that important value into an s-register before you call a function!!

Stack Push and Pop

Pushing into Stack

- First: Adjust stack pointer “up” (i.e., to a **smaller** address)
- Next: Copy values *into the* stack

Popping from Stack

- First: Copy values *from the* stack
- Next: Adjust stack pointer “down” (i.e., to a **larger** address)



Stack Push

PUSH INTO STACK:

1. Adjust stack pointer
2. Copy values into the stack

1. Adjust stack pointer “up”

- Subtract from the **\$sp** register **4 times** the number of registers to be pushed (why 4 times??)

2. Copy values into stack

- **EXAMPLE:** let's say we want to store 2 registers' data into the stack...
- To write TO the stack (i.e. in memory), we should use the **sw** instruction

```
addiu $sp, $sp, -8    # 2 registers to save
sw $s0, 4($sp)
sw $s1, 0($sp)        # See blackboard...
```

Why **addiu**?

Because there is no such thing as a negative memory address

AND

we want to avoid triggering a processor-level **exception on overflow**



Stack Pop

POP OUT OF STACK:

1. Copy values from the stack
2. Adjust stack pointer



1. Copy values from stack

- **EXAMPLE:** let's say we want to copy 2 words from the stack...
- To write FROM the stack (i.e. in memory), to regs we should use the **lw** instruction

2. Adjust stack pointer “down”

- **ADD** to the **\$sp** register **4 times** the number of words to be popped
(why 4 times??)

```
lw $s0, 4($sp)           # See blackboard...
```

```
lw $s1, 0($sp)
```

```
addiu $sp, $sp, 8         # 2 registers to save
```

Next on CS 64.....!!!

*What happens when we have function calls **to other functions**??*

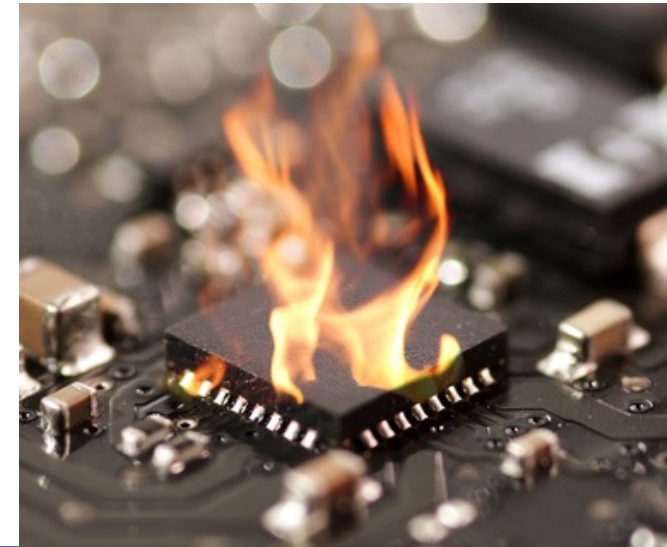
Will the “Calling Convention”, as it stands now, be enough to help us out??

Can the CPU handle this madness???

Should I switch my major to “Basket Weaving”

(or worse, Physics!!!) & leave all this craziness behind me???

COMING UP NEXT: the Shocking Conclusion!!!



An Example...

Consider when *functionX()* calls *functionY()*

functionX:

uses \$s0, \$s1, \$s2, \$s3

Then calls functionY(42)

li \$a0, 42

jal functionY

functionY:

DOES NOT know what \$s regs. **functionX** used

(*this is the typical case*)

but it DOES know what \$s regs. **it** plans on using

(example: it plans on using \$s0, \$s1 only)

Plan for functionY:

1. **PUSH** \$s0, \$s1 **only** into stack (i.e., saves the “current” values for the caller code)
2. **Runs** its code/calculations/instructions (where \$s0, \$s1 get overwritten)
3. **POPS** \$s0, \$s1 back from stack (i.e., recovers the old values for the caller code)
4. **Returns** to functionX via **jr \$ra**

Why? Because it plans on using/overwriting those 2 s-registers itself

```
int main() {
    second()
    return 0; }

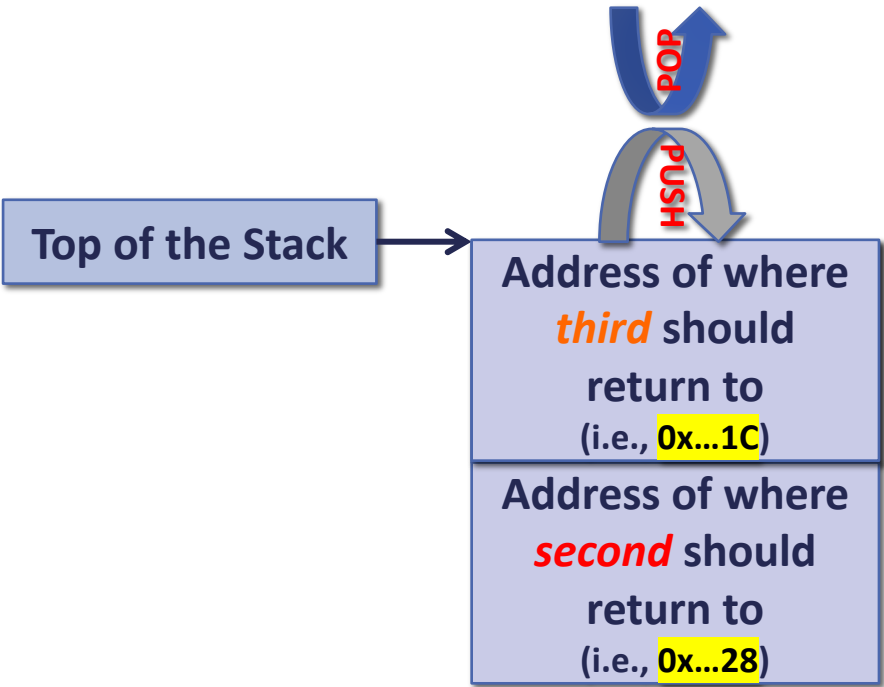
void second(){
    third ();
    return; }

void third() {
    fourth ();
    return; }

void forth() {
    return; }
```

MIPS Call Stack

After “jal second”, \$ra = 0x...28
After “jal third”, \$ra = 0x...1C
After “jal fourth”, \$ra = 0x...0C



pretend addresses → 0x...00 **fourth:**
jr \$ra

third:
0x...04 *push \$ra in stack*
0x...08 **jal fourth**
0x...0C *pop \$ra from stack*
0x...10 **jr \$ra**

second:
0x...14 *push \$ra in stack*
0x...18 **jal third**
0x...1C *pop \$ra from stack*
0x...20 **jr \$ra**

main:
0x...24 **jal second**

0x...28 **li \$v0, 10**
0x...2C **syscall**

MIPS Call Stack

fourth:

0x...00 `jr $ra`

third:

0x...04 *push \$ra in stack*

0x...08 `jal fourth`

0x...0C *pop \$ra from stack*

0x...10 `jr $ra`

second:

0x...14 *push \$ra in stack*

0x...18 `jal third`

0x...1C *pop \$ra from stack*

0x...20 `jr $ra`

main:

0x...24 `jal second`

0x...28 `li $v0, 10`

0x...2C `syscall`

Top of the Stack →

Step 0 (0x...24)

Top of the Stack →

Address of where
second should
return to
(i.e., 0x...28)



Step 1 (0x...14)

Top of the Stack →

Address of where
third should
return to
(i.e., 0x...1C)



Address of where
second should
return to
(i.e., 0x...28)

Step 2 (0x...04)

Top of the Stack →

Address of where
second should
return to
(i.e., 0x...28)



Step 3 (0x...0C)

Top of the Stack →



Step 4 (0x...1C)

fourth:

jr \$ra

*Actual
MIPS
assembly*

third:

addiu \$sp, \$sp, -4

sw \$ra, 0(\$sp)

jal fourth

lw \$ra, 0(\$sp)

addiu \$sp, \$sp, 4

jr \$ra

second:

addiu \$sp, \$sp, -4

sw \$ra, 0(\$sp)

jal third

lw \$ra, 0(\$sp)

addiu \$sp, \$sp, 4

jr \$ra

main:

jal second

li \$v0, 10

syscall

fourth:

jr \$ra

*Pseudo-
assembly*

third:

push \$ra

jal fourth

pop \$ra

jr \$ra

second:

push \$ra

jal third

pop \$ra

jr \$ra

main:

jal second

li \$v0, 10

syscall

Example...

```

...
...
int subTwo(int a, int b)
{
    int sub = a - 2*b;
    return sub;
}

int doSomething(int x, int y)
{
    int a = subTwo(x, y);
    int b = subTwo(y, x);
    return a + b;
}
...
...

```

subTwo doesn't call anything

What (registers) should I map **a** and **b** to? *\$a0 and \$a1*

What else is going on? *There's a returned value. It should be placed in \$v0*

doSomething DOES call a function (it calls subTwo twice!)

What should I expect args **x** and **y** to be initially? *\$a0 and \$a1*

Is that ok? subTwo seems to need to use those regs...

Ask yourself: do we need to preserve them at all? (in this case, yes – **why?**)

*Since we want to preserve them across the function call,
we should copy them to \$s0 and \$s1 first*

What registers should I map **a** and **b** to?

Ask yourself: do we need to preserve them at all?

"a+b" has to eventually be \$v0.

*I use **a** in the 1st line – it comes back as \$v0,
but then I call function in the 2nd line... is that a problem...?*

What about **b**?

We need to preserve a, so copy it over to \$s2
We don't need to preserve b

subTwo:

```
sll $a1, $a1, 1
sub $v0, $a0, $a1
jr $ra
```

doSomething:

```
# Note 1: Something CALLED doSomething.
# Note 2: doSomething NEEDS to preserve
# at least $s0-$s2 AND $ra
# That's 4 regs to preserve - where?
# IN THE STACK!!!
```

```
addiu $sp, $sp, -16
sw $s0, 0($sp)
sw $s1, 4($sp)
sw $s2, 8($sp)
sw $ra, 12($sp)
```

```
move $s0, $a0 # can now use $s0=x
move $s1, $a1 # can now use $s1=y
```

```
jal subTwo      # returns a in $v0
```

```
move $s2, $v0 # can now use $s2=a
```

```
move $a0, $s1 # switch around vars: y=$a0
move $a1, $s0 # switch around vars: x=$a1
```

```
jal subTwo      # returns b in $v0
```

```
add $v0, $v0, $s2 # return is $v0=a+b
```

Done!

```
# Now we need to pop from the stack
# the old preserved values back to their
# original s-regs (also, the $ra!)
# and then return to the caller
```

```
lw $s0, 0($sp)
lw $s1, 4($sp)
lw $s2, 8($sp)
lw $ra, 12($sp)
addiu $sp, $sp, 16

jr $ra
```

```
int subTwo(int a, int b)
{
    int sub = a - 2*b;
    return sub;
}

int doSomething(int x, int y)
{
    int a = subTwo(x, y);
    int b = subTwo(y, x);
    return a + b; }
}
```

Up Next!

Recursive Functions!!!



To Dos

Study for **Quiz #3 on Wednesday!**

- Lecture **8** – Memory Access
- Lecture **9** – Memory Addressing, Instruction Assembly/Disassembly
- Lecture **10** – Basic Function Calls
- Lecture **11** – Functions Calling Functions, MIPS C.C., How the Stack Works
- **Reading:** CS64_Handouts 3, 4

Current assignment is due on Monday

Start reading the next handout (**Handout 5**) – must complete it by Monday!

Remember: **Lab on Friday**

</LECTURE>